

SIMATS ENGINEERING

MLA0201-FUNDAMENTALS OF MACHINE LEARNING

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Student Name:Shaik Mubarak

Reg No:192425194

Q1. OPTIMIZING A LINEAR REGRESSION MODEL FOR HOUSE PRICE PREDICTION

Problem Identification

A linear regression model with high prediction error may be affected by multicollinearity, outliers, irrelevant features, missing values, or poor feature selection. For example, house area and number of rooms may be highly correlated. Extreme house prices can also influence the regression line significantly.

Debugging and Analysis

First, check missing values and descriptive statistics. Correlation analysis can be used to identify highly correlated independent variables. The Variance Inflation Factor (VIF) can also be used to detect multicollinearity. Outliers can be detected using boxplots or the IQR method. Model performance can be evaluated using Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 score.

Optimization

The model can be improved by:

- Removing or combining highly correlated features.
- Removing extreme outliers when they are data errors.
- Adding useful features such as location, number of bedrooms, age of property, and parking availability.
- Applying feature scaling when required.
- Using cross-validation to check generalization.

Example Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```

df = pd.read_csv("house_data.csv")
df = df.dropna()
X = df.drop("price", axis=1)
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LinearRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("RMSE:", mean_squared_error(y_test, pred) ** 0.5)
print("R2 Score:", r2_score(y_test, pred))

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

X = np.array([
    [1000, 2, 5],
    [1200, 2, 8],
    [1500, 3, 10],
    [1800, 3, 12],
    [2000, 4, 15],
    [2200, 4, 18],
    [2500, 5, 20],
    [2800, 5, 22],
    [3000, 6, 25],
    [3500, 6, 28]
])

y = np.array([

```

```

50, 60, 75, 90, 105,
115, 135, 150, 165, 190
])
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("House Price Prediction")
print("-----")
print("Actual Prices:", y_test)
print("Predicted Prices:", np.round(pred, 2))
print("RMSE:", round(np.sqrt(mean_squared_error(y_test, pred)), 2))
print("R2 Score:", round(r2_score(y_test, pred), 2))

```

Expected Improvement

After removing problematic data, selecting relevant features, and scaling the inputs, the model should produce lower RMSE and higher R^2 , indicating better prediction performance.

Q2. IMPROVING SPAM EMAIL CLASSIFICATION USING A LINEAR MODEL

Problem Identification

Spam emails may be incorrectly classified as non-spam because of an improper decision boundary, imbalanced training data, weak features, or an unsuitable classification threshold. Missing important words or patterns from the feature set can also cause misclassification.

Debugging and Analysis

The confusion matrix should be examined to identify false negatives, where spam emails are predicted as non-spam. Precision, recall, and F1-score should be calculated. For spam detection, recall is especially important, because a false negative means that a spam email reaches the user.

Optimization

The model can be improved by:

- Adjusting the classification threshold.
- Extracting useful text features using TF-IDF.
- Balancing spam and non-spam samples.
- Retraining the model using balanced data.
- Removing irrelevant features.

Code:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

emails = [
    "win free money now",
    "claim your free prize",
    "you won a lottery",
    "free gift waiting for you",
    "click here to win prize",
    "meeting at ten tomorrow",
    "submit your assignment today",
    "project discussion tomorrow",
    "class starts at nine",
    "please send the report",
    "team meeting at three",
    "complete the project"
]

labels = [
    1, 1, 1, 1, 1,
    0, 0, 0, 0, 0, 0, 0
```

```

]
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(emails)
X_train, X_test, y_train, y_test = train_test_split(
    X, labels, test_size=0.3, random_state=42, stratify=labels
)
model = LogisticRegression(class_weight="balanced")
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Spam Email Classification")
print("-----")
print(classification_report(
    y_test, pred, zero_division=0
))

```

Expected Improvement

Balanced training and better text features improve the decision boundary and increase the model's ability to detect spam, particularly improving spam recall and F1-score.

Q3. IMPROVING NAÏVE BAYES USING LAPLACE SMOOTHING

Problem Identification

Naïve Bayes calculates probabilities based on the training data. If a particular feature value never occurs with a class, its probability can become zero. Multiplying probabilities during classification can then produce an overall probability of zero.

Missing values can also result in unreliable probability estimates.

Debugging and Analysis

The conditional probability table should be inspected for zero values. For example:

$$P(\text{word} = \text{"offer"} \mid \text{spam}) = 0$$

If this probability becomes zero, the complete probability calculation may become zero even when other features strongly indicate spam.

Optimization

Laplace smoothing can solve the zero-frequency problem by adding a small value to each frequency:

$$[P(x|C)=\frac{\text{count}(x,C)+1}{\text{count}(C)+N}]$$

where N represents the number of possible feature values.

Missing values should also be handled by removing or appropriately imputing them.

Code:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

emails = [
    "free money offer",
    "win free prize",
    "free gift offer",
    "meeting tomorrow",
    "project meeting today",
    "submit assignment",
    "win money prize",
    "class meeting"
]

labels = [1, 1, 1, 0, 0, 0, 1, 0]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(emails)

X_train, X_test, y_train, y_test = train_test_split(
    X, labels, test_size=0.25, random_state=42
)

model = MultinomialNB(alpha=1.0)

model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
print("Naive Bayes with Laplace Smoothing")
print("-----")
print("Actual:", y_test)
print("Predicted:", pred)
print("Accuracy:", round(accuracy_score(y_test, pred), 2))
```

Here, $\alpha=1.0$ applies **Laplace smoothing**.

Expected Improvement

Smoothing prevents zero probabilities and makes the classifier more robust when unseen feature combinations occur in testing data.

Q4. IMPROVING A POOR DISCRIMINANT FUNCTION

Problem Identification

A discriminant function may fail to separate two classes when:

- Features strongly overlap.
- Feature weights are incorrect.
- Features have different scales.
- Irrelevant features are included.
- The chosen transformation is unsuitable.

Debugging and Analysis

The decision function can be examined to understand how each feature contributes to classification. A decision boundary can be visualized for two-dimensional data.

If the classes overlap heavily, many samples will be located close to the boundary, causing classification errors.

Optimization

The discriminant function can be improved by:

- Adjusting feature weights.
- Normalizing or standardizing features.
- Removing irrelevant features.
- Applying suitable feature transformations.

- Selecting better discriminative features.

Code:

```
import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

from sklearn.metrics import accuracy_score

X = np.array([
    [2, 3], [3, 4], [4, 5], [5, 6],
    [6, 7], [7, 8], [8, 9], [9, 10],
    [10, 11], [11, 12], [12, 13], [13, 14]
])

y = np.array([
    0, 0, 0, 0, 0, 0,
    1, 1, 1, 1, 1, 1
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

model = LinearDiscriminantAnalysis()

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Discriminant Analysis")

print("-----")

print("Actual:", y_test)

print("Predicted:", pred)
```



```
print("Accuracy:", accuracy_score(y_test, pred))
```

Expected Improvement

Normalization and appropriate feature selection can create a more reliable decision boundary and improve class separation and accuracy.

Q5. REDUCING OVERFITTING IN BAYESIAN LOGISTIC REGRESSION

Problem Identification

Overfitting occurs when a model learns the training data too closely and performs poorly on unseen data.

Common signs include:

- Very high training accuracy.
- Much lower testing accuracy.
- Low training error but high testing error.
- Large or unstable model coefficients.

Debugging and Analysis

Training and testing performance should be compared.

For example:

Training Accuracy = 98%

Testing Accuracy = 75%

This large difference indicates that the model is overfitting.

Cross-validation can also be used to check whether the model performs consistently across different subsets.

Optimization

Overfitting can be reduced using:

- **L1 regularization**, which can make some coefficients zero.
- **L2 regularization**, which reduces excessively large coefficients.
- Cross-validation.
- Feature selection.
- Removing irrelevant features.
- Increasing training data.

Code:

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = make_classification(
    n_samples=200,
    n_features=10,
    n_informative=5,
    n_redundant=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(
    penalty="l2",
    C=0.1,
    max_iter=1000
)

model.fit(X_train, y_train)

train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

print("Logistic Regression with L2 Regularization")
```

```
print("-----")
print("Training Accuracy:",
      round(accuracy_score(y_train, train_pred), 2))
print("Testing Accuracy:",
      round(accuracy_score(y_test, test_pred), 2))
```

Expected Improvement

Regularization controls model complexity and reduces dependence on individual training samples, resulting in better generalization and improved testing performance.

Q6. FEATURE SCALING IN SALES PREDICTION

Problem Identification

In a sales prediction model, features such as advertising budget in thousands and number of outlets may have very different numerical ranges.

For example:

- Advertising budget: 10–1000
- Number of outlets: 1–50

The large-scale feature can dominate calculations and make optimization less efficient.

Debugging and Analysis

Unscaled features can result in regression coefficients having very different magnitudes. In gradient-based optimization, the model may take inefficient steps toward the minimum.

This can lead to slower convergence and difficulty in comparing feature importance.

Optimization

Two common approaches are:

Standardization:

$$[z = \frac{x - \mu}{\sigma}]$$

Normalization:

$$[x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}]$$

Standardization is often useful when features have different distributions.

Code:

```
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

X = np.array([
    [10, 2],
    [20, 4],
    [30, 5],
    [40, 7],
    [50, 8],
    [60, 10],
    [70, 12],
    [80, 14],
    [90, 15],
    [100, 18]
])

y = np.array([
    25, 35, 45, 55, 65,
    78, 88, 100, 112, 125
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LinearRegression()
```

```
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Sales Prediction using Feature Scaling")
print("-----")
print("Actual Sales:", y_test)
print("Predicted Sales:", np.round(pred, 2))
print("R2 Score:", round(r2_score(y_test, pred), 2))
print("Coefficients:", np.round(model.coef_, 2))
```

Expected Improvement

Scaling puts features on a comparable range, improves numerical stability, and can make optimization faster and more effective.

Q7. HANDLING CLASS IMBALANCE IN FRAUD DETECTION

Problem Identification

Fraud detection datasets usually contain many legitimate transactions and relatively few fraudulent transactions.

For example:

- Legitimate transactions = 98%
- Fraudulent transactions = 2%

A model can achieve 98% accuracy by predicting almost everything as legitimate, while failing to detect fraud.

Debugging and Analysis

Accuracy alone is not sufficient. The model should be evaluated using:

- **Precision**
- **Recall**
- **F1-score**
- Confusion matrix

For fraud detection, recall is important because missing a fraudulent transaction is costly.

Optimization

Class imbalance can be handled using:

- Random oversampling.
- Random undersampling.
- SMOTE.
- Class weighting.
- Threshold adjustment.

Code:

```
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

X = np.array([
    [10, 100], [20, 200], [15, 150], [30, 300],
    [25, 250], [40, 400], [35, 350], [45, 450],
    [50, 500], [55, 550], [60, 600], [65, 650],
    [70, 700], [75, 750], [80, 800],
    [85, 850], [90, 900], [95, 950],
    [5, 50], [8, 80]
])

y = np.array([
    0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 0, 0, 0, 0,
    0, 0, 0, 0,
    1, 1
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

model = LogisticRegression()
```

```

class_weight="balanced",
max_iter=1000
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Fraud Detection")
print("-----")
print(classification_report(
    y_test, pred, zero_division=0
))

```

Expected Improvement

Class weighting gives more importance to fraudulent samples. This generally improves fraud recall and F1-score, making the model more useful for detecting minority-class cases.

Q8. IMPROVING NAÏVE BAYES WITH CORRELATED FEATURES

Problem Identification

Naïve Bayes assumes that features are conditionally independent given the class. Highly correlated features violate this assumption.

For example, annual income and monthly income contain similar information. Naïve Bayes may effectively count the same information more than once.

Debugging and Analysis

Feature relationships can be examined using:

- Correlation matrices.
- Heatmaps.
- Pair plots.
- Statistical feature analysis.

If two or more features have very high correlation, redundant information may be present.

Optimization

The model can be improved using:

- Feature selection.
- Removing redundant features.
- Principal Component Analysis (PCA).
- Dimensionality reduction.
- Selecting the most informative variables.

Code:

```
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

X = np.array([
    [10, 20, 1], [12, 24, 1], [15, 30, 2],
    [18, 36, 2], [20, 40, 2], [22, 44, 3],
    [50, 100, 8], [55, 110, 9], [60, 120, 10],
    [65, 130, 11], [70, 140, 12], [75, 150, 13]
])

y = np.array([
    0, 0, 0, 0, 0, 0,
    1, 1, 1, 1, 1, 1
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

pca = PCA(n_components=2)
```



```
X_train = pca.fit_transform(X_train)
X_test = pca.transform(X_test)
model = GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Naive Bayes with PCA")
print("-----")
print("Actual:", y_test)
print("Predicted:", pred)
print("Accuracy:", accuracy_score(y_test, pred))
```

Expected Improvement

Removing redundant features or transforming correlated features using PCA can reduce the effect of the independence-assumption violation and improve classification performance.

Q9. IMPROVING LOGISTIC REGRESSION WITH OVERLAPPING CLASSES

Problem Identification

Logistic regression creates a linear decision boundary. If two classes have strongly overlapping feature distributions, a straight line may not separate them effectively.

This causes:

- False positives.
- False negatives.
- Low classification accuracy.
- Poor precision and recall.

Debugging and Analysis

The feature space should be visualized to identify overlapping regions. The confusion matrix should also be examined to determine which class is frequently misclassified.

If the data cannot be separated using a straight boundary, feature transformation may be required.

Optimization

The model can be improved using:

- Polynomial features.
- Logarithmic or other transformations.
- Feature selection.
- Standardization.
- L1/L2 regularization.

Polynomial features allow logistic regression to represent nonlinear relationships.

Code:

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = make_moons(
    n_samples=200,
    noise=0.2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = make_pipeline(
    PolynomialFeatures(degree=2),
    StandardScaler(),
    LogisticRegression()
)

model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Logistic Regression with Polynomial Features")
```

```
print("-----")
print("Actual:", y_test)
print("Predicted:", pred)
print("Accuracy:", round(accuracy_score(y_test, pred), 2))
```

Expected Improvement

Polynomial transformation allows the model to learn nonlinear boundaries, improving its ability to separate overlapping classes while regularization prevents excessive complexity.

Q10. IMPROVING AN UNDERFITTING DISCRIMINANT ANALYSIS MODEL

Problem Identification

Underfitting occurs when a model is too simple to capture the important patterns in the dataset.

Signs include:

- Low training accuracy.
- Low testing accuracy.
- High training and testing error.
- Poor representation of complex relationships.

This indicates high bias.

Debugging and Analysis

The model should be evaluated on both training and testing datasets.

For example:

Training Accuracy = 65%

Testing Accuracy = 62%

Since both accuracies are low, the model is likely underfitting rather than overfitting.

Feature limitations should also be analyzed to determine whether important information is missing.

Optimization

The model can be improved by:

- Adding relevant features.
- Removing weak features.

- Applying nonlinear transformations.
- Increasing model complexity.
- Using polynomial features.
- Applying suitable dimensionality-reduction techniques.

Code:

```

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis

from sklearn.pipeline import make_pipeline

from sklearn.metrics import accuracy_score

data = load_iris()

X = data.data

y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = make_pipeline(
    StandardScaler(),
    QuadraticDiscriminantAnalysis()
)

model.fit(X_train, y_train)

train_pred = model.predict(X_train)

test_pred = model.predict(X_test)

print("Discriminant Analysis - Underfitting Optimization")

print("-----")

print("Training Accuracy:",
      round(accuracy_score(y_train, train_pred), 2))

print("Testing Accuracy:",

```

```
round(accuracy_score(y_test, test_pred), 2))
```

Expected Improvement

Adding useful features and increasing model flexibility allows the model to capture more complex patterns. This reduces bias and improves both training and testing performance.